Manual de Instalación

FaceAttend EDU — Plataforma de gestión de asistencia mediante reconocimiento biométrico

Proyecto Formativo

Tecnólogo en Análisis y Desarrollo de Software — SENA

Nombre del proyecto	FaceAttend EDU
Tipo de documento	Manual de instalación
Versión del documento	1.0
Fecha de elaboración	06 de octubre de 2026
Dirigido a	Técnico o persona responsable de poner en marcha el sistema
Repositorio	github.com/Jonas7891/ProyectoFaceAttendEDU
Programa de formación	Análisis y Desarrollo de Software (ADSO)
Ficha	_____
Instructor(a)	_____
Centro de formación	_____
Integrantes	_____

Alcance de este documento: instalación y puesta en marcha del sistema en un equipo local o servidor. No cubre el uso funcional del sistema (Manual de usuario), la construcción interna del código (Manual técnico) ni la administración operativa (Manual de administrador).

Control de versiones

Versión	Fecha	Descripción del cambio	Responsable
1.0	06/10/2026	Creación inicial del manual de instalación. Documenta el despliegue del stack completo (18 contenedores) con Docker Compose, la configuración mediante el archivo `.env` único, la cadena de migraciones Liquibase y el procedimiento de verificación.	Equipo desarrollador

> **Cómo mantener esta tabla.** Cada vez que el procedimiento de instalación cambie (nuevo servicio, nueva variable de entorno, cambio de puerto o de versión de una herramienta), se agrega una fila nueva: se incrementa la versión, se indica la fecha, se describe qué cambió y quién lo hizo. Un manual sin control de versiones no permite saber si corresponde a la versión del software que se está instalando.

Documentos relacionados

Documento	Responde a la pregunta
Manual de usuario	¿Cómo uso el sistema?
Manual técnico	¿Cómo está construido el sistema?
Manual de instalación (este documento)	¿Cómo instalo y ejecuto el sistema?
Manual de administrador	¿Cómo administro el sistema?

Tabla de contenido

1. Introducción

1.1 Objetivo · 1.2 Alcance · 1.3 Público objetivo · 1.4 Cómo leer este manual

2. Descripción general del sistema a instalar

2.1 Qué se instala · 2.2 Componentes del stack · 2.3 Diagrama de despliegue

3. Requisitos previos

3.1 Requisitos de hardware · 3.2 Sistema operativo · 3.3 Software requerido · 3.4 Herramientas opcionales · 3.5 Puertos requeridos · 3.6 Conectividad

4. Descarga e instalación

4.1 Instalar Docker Desktop · 4.2 Instalar Git · 4.3 Obtener el proyecto · 4.4 Verificar la estructura

5. Configuración inicial

5.1 Crear el archivo `.env` · 5.2 Variables de entorno · 5.3 Casos de personalización · 5.4 Canal de correo (opcional)

6. Puesta en marcha

6.1 Levantar el stack · 6.2 Orden de arranque · 6.3 Base de datos y migraciones · 6.4 Datos iniciales · 6.5 Datos de prueba (opcional)

7. Verificación de la instalación

7.1 Lista de verificación · 7.2 Estado de contenedores · 7.3 Pruebas de servicios · 7.4 Prueba de autenticación · 7.5 Prueba desde el navegador · 7.6 Aislamiento de red

8. Instalación de la aplicación móvil (opcional)

9. Operación cotidiana

9.1 Comandos frecuentes · 9.2 Detener y reiniciar · 9.3 Actualizar · 9.4 Desinstalar

10. Solución de problemas (*troubleshooting*)

11. Instalación sin Docker (avanzado)

12. Glosario

13. Soporte y referencias

14. Anexos

A. Mapa de puertos · B. Rutas de la API · C. Usuarios iniciales · D. Formato de reporte de incidencias

1. Introducción

> Este manual explica, paso a paso, cómo preparar el entorno, configurar las dependencias y poner en funcionamiento **FaceAttend EDU** en un equipo local o en un servidor, partiendo de una máquina sin nada instalado.

FaceAttend EDU es una plataforma web y móvil para la gestión de asistencia en instituciones educativas. Automatiza el registro de entrada y salida mediante reconocimiento facial, con alternativa de registro manual, centraliza los históricos de asistencia y gestiona justificaciones, reportes y alertas por rol.

1.1 Objetivo

Permitir que cualquier persona con conocimientos técnicos básicos instale y ponga en funcionamiento el sistema completo siguiendo instrucciones claras, completas y verificables, sin necesidad de consultar al equipo desarrollador.

1.2 Alcance

Este manual SÍ cubre	Este manual NO cubre
• Requisitos de hardware y software. • Instalación de las herramientas base (Docker, Git). • Obtención del código del proyecto. • Configuración de variables de entorno. • Despliegue del stack completo con Docker Compose. • Creación de la base de datos y ejecución de migraciones. • Verificación funcional de la instalación. • Solución de los problemas más frecuentes. • Instalación de la aplicación móvil en modo desarrollo.	• El uso funcional del sistema → *Manual de usuario*. • La arquitectura interna y el código → *Manual técnico*. • La gestión de usuarios, roles y reportes → *Manual de administrador*. • El despliegue en un entorno de producción con dominio público, certificados TLS y alta disponibilidad. • La configuración de la integración continua (CI/CD).

1.3 Público objetivo

Este documento está dirigido al **técnico o persona responsable de poner en marcha el sistema**: un instructor que lo despliega para evaluarlo, un integrante del equipo que configura su equipo de trabajo, o un administrador de sistemas que lo instala en un servidor institucional.

Se asume que la persona sabe abrir una terminal y ejecutar comandos. **No** se asume conocimiento previo de Docker, de microservicios ni de ninguno de los lenguajes del proyecto: todo lo necesario se explica aquí, y los términos técnicos se definen en el **Glosario** (sección 12).

1.4 Cómo leer este manual

Las secciones están en el orden en que deben ejecutarse. Para una instalación nueva, siga las secciones 3 a 7 de forma consecutiva; las secciones 8 en adelante son complementarias y se consultan según la necesidad.

A lo largo del documento se usan tres tipos de aviso:

> **Nota.** Información adicional que ayuda a entender el paso, pero cuya omisión no rompe la instalación.

> **Atención.** Paso donde es frecuente equivocarse, o acción que puede causar pérdida de datos. Léalo antes de ejecutar el comando.

> **Verificación.** Resultado concreto que debe observarse para confirmar que el paso anterior salió bien. Si no coincide, no continúe: consulte la sección 10.

Los bloques con fondo gris son **comandos**: se escriben en la terminal tal como aparecen. Las líneas que empiezan con `#` son comentarios explicativos y no se ejecutan.

2. Descripción general del sistema a instalar

> Conviene entender qué se va a instalar antes de instalarlo: FaceAttend EDU no es un programa único, sino un conjunto de servicios que se levantan y coordinan entre sí.

2.1 Qué se instala

El sistema está construido como una arquitectura de **microservicios**: en lugar de un solo programa grande, son nueve servicios independientes, cada uno responsable de una parte del negocio (identidad, permisos, académico, horarios, asistencia, biometría, configuración, notificaciones y calidad). Delante de ellos hay una **puerta de enlace** (*API Gateway*) que recibe todas las peticiones y las dirige al servicio correcto, y detrás están las bases de datos y el sistema de mensajería.

Instalar todo esto a mano sería inviable: cada servicio está escrito en un lenguaje distinto (Java, TypeScript, Python y Go) y cada uno tiene sus propias dependencias. Por eso el proyecto usa **Docker**, que empaqueta cada servicio con todo lo que necesita dentro de un *contenedor*, y **Docker Compose**, que levanta los 18 contenedores en el orden correcto con un único comando.

> **La consecuencia práctica es importante:** no necesita instalar Java, Node.js, Python ni Go en su equipo. Docker los descarga y los ejecuta dentro de los contenedores. Solo necesita Docker y Git. La instalación manual de los lenguajes se describe en la sección 11, y solo es necesaria si va a modificar el código.

2.2 Componentes del stack

El despliegue completo levanta **18 contenedores**, agrupados en tres capas:

Capa de presentación

Contenedor	Tecnología	Puerto	Función
`frontend-web`	Expo Web + nginx	8090	Interfaz gráfica que carga el navegador (aplicación de página única).

Capa de aplicación

Contenedor	Tecnología	Puerto	Función
`kong-gateway`	Kong OSS 3.6	8080 / 8001	Puerta de enlace: enruta `/api/*`, aplica CORS y límite de peticiones.
`ms-identity`	Java 21 + Spring Boot	8081	Personas, usuarios, credenciales y sesiones.
`ms-authorization`	Java 21 + Spring Boot	8082	Roles y permisos (RBAC).

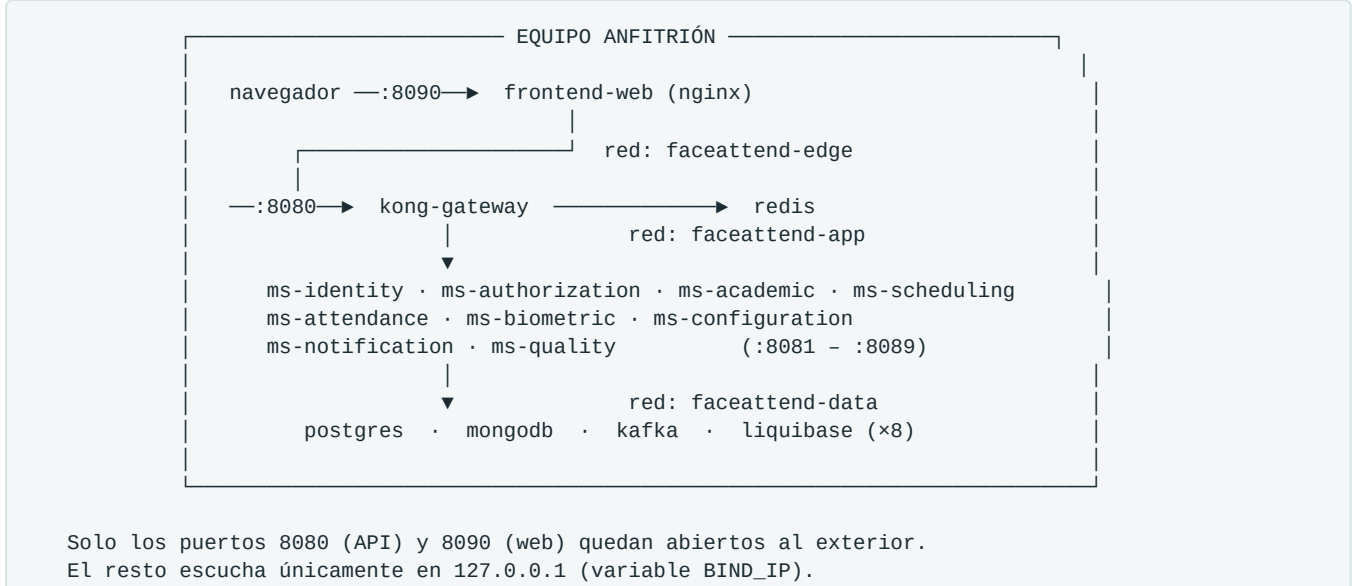
Contenedor	Tecnología	Puerto	Función
`ms-academic`	TypeScript + Fastify	8083	Sedes, programas, periodos, fichas, cursos y matrículas.
`ms-scheduling`	Java 21 + Spring Boot	8084	Ambientes, bloques de horario y sesiones de clase.
`ms-attendance`	Java 21 + Spring Boot	8085	Registros de asistencia y justificaciones.
`ms-biometric`	Python 3.12 + FastAPI	8086	Vectores biométricos (facial y dactilar).
`ms-configuration`	TypeScript + Fastify	8087	Parámetros académicos y de seguridad.
`ms-notification`	Go 1.22 + Gin	8088	Alertas y envío de correo.
`ms-quality`	TypeScript + Fastify	8089	Instrumento de calidad ISO/IEC 25010.
`redis`	Redis 7	6379	Almacén del límite de peticiones de Kong.

Capa de datos

Contenedor	Tecnología	Puerto	Función
`postgres`	PostgreSQL 17	5432	Base de datos principal `faceattend_db` con 8 esquemas, uno por contexto.
`mongodb`	MongoDB 7	27017	Almacenamiento de los vectores biométricos.
`kafka`	Kafka 3.8 (KRaft)	9092	Mensajería de eventos de dominio entre servicios.
`*-liquibase` (8)	Liquibase 4.29	—	Ocho tareas temporales que crean las tablas y cargan los datos iniciales. Se ejecutan, terminan y se detienen.

2.3 Diagrama de despliegue

Los contenedores se reparten en tres redes aisladas. El resultado es que la capa de datos es **inalcanzable** desde la red que ve el navegador:



Solo los puertos 8080 (API) y 8090 (web) quedan abiertos al exterior.
El resto escucha únicamente en 127.0.0.1 (variable BIND_IP).

Para el instalador esto significa: una vez levantado el stack, usted solo necesita dos direcciones: `http://localhost:8090` para la interfaz y `http://localhost:8080` para la API. Las bases de datos no se exponen y no debe abrirlas al exterior.

3. Requisitos previos

> Verifique todos los requisitos de esta sección **antes** de descargar el proyecto. Un requisito no cumplido provoca fallos difíciles de diagnosticar más adelante.

3.1 Requisitos de hardware

El stack levanta 18 contenedores simultáneos, cuatro de ellos con máquina virtual de Java. Es un consumo de memoria considerable y es el requisito que más frecuentemente falla.

Recurso	Mínimo	Recomendado	Observación
Memoria RAM	**8 GB**	**16 GB**	Con 8 GB el stack arranca, pero conviene cerrar otras aplicaciones. Docker Desktop debe tener al menos 6 GB asignados.
Procesador	4 núcleos (Intel Core i5 / AMD Ryzen 5 o equivalente)	8 núcleos	Debe soportar virtualización (VT-x / AMD-V) y tenerla habilitada en la BIOS.
Espacio en disco	**20 GB libres**	30 GB libres	Imágenes de Docker (~12 GB), código fuente, dependencias y volúmenes de datos.
Red	Conexión a internet	Banda ancha	Necesaria solo durante la primera construcción, para descargar imágenes y dependencias (~5 GB).

Atención — virtualización. Docker no funciona si la virtualización por hardware está deshabilitada. En Windows, verifíquelo en el Administrador de tareas → pestaña *Rendimiento* → *CPU*: debe decir **Virtualización: Habilitada**. Si dice deshabilitada, actívela en la BIOS/UEFI del equipo (opción *Intel VT-x*, *SVM Mode* o *Virtualization Technology*).

3.2 Sistema operativo

Sistema operativo	Versión mínima	Observación
Windows 10 / 11	Build 19044 o superior, 64 bits	Requiere WSL 2 (se instala junto con Docker Desktop).
Linux Ubuntu	22.04 LTS o superior	Docker Engine + plugin Compose. Otras distribuciones compatibles: Debian 12, Fedora 39.
macOS	13 (Ventura) o superior	Compatible con procesadores Intel y Apple Silicon (M1/M2/M3).

3.3 Software requerido

Solo se necesitan dos herramientas. Docker aporta todo lo demás.

Herramienta	Versión mínima	Dónde obtenerla	Comando de verificación	Para qué sirve
Docker	24.0 o superior	docker.com/get-started	`docker --version`	Ejecuta cada servicio dentro de su propio contenedor.
Docker Compose	2.20 o superior	Incluido en Docker Desktop	`docker compose version`	Levanta y coordina los 18 contenedores.
Git	2.40 o superior	git-scm.com	`git --version`	Descarga el código del repositorio.

Docker Compose: dos formas de escribirlo. Las versiones modernas usan `docker compose` (dos palabras, es un sub-comando de Docker). Las versiones antiguas usaban `docker-compose` (con guion, programa aparte). Este manual usa siempre la forma moderna. Si su equipo solo reconoce la forma con guion, actualice Docker: el proyecto usa la directiva `include`, que requiere Compose 2.20 o superior.

3.4 Herramientas opcionales

Herramienta	Cuándo es útil	Uso
Visual Studio Code	Si va a revisar o editar el código	Editor con soporte para los cuatro lenguajes del proyecto.
Postman o Insomnia	Si va a probar la API manualmente	Enviar peticiones HTTP sin escribir comandos `curl`.
DBeaver o pgAdmin	Si necesita consultar la base de datos	Cliente gráfico para PostgreSQL.
MongoDB Compass	Si necesita revisar los vectores biométricos	Cliente gráfico para MongoDB.
Mailpit	Si va a probar el envío de correos	Buzón de correo falso para desarrollo (sección 5.4).

3.5 Puertos requeridos

El sistema necesita **15 puertos libres**. Si alguno está ocupado por otro programa, el contenedor correspondiente no arrancará.

Puerto	Servicio	Expuesto a	Conflicto frecuente con
8080	Kong (API)	Toda la red (0.0.0.0)	Tomcat, Jenkins, otro servidor de aplicaciones.
8090	Interfaz web	Toda la red (0.0.0.0)	Poco frecuente.
8001	Kong (administración)	Solo local (127.0.0.1)	Poco frecuente.
8081–8089	Los nueve microservicios	Solo local (127.0.0.1)	8081 y 8085 suelen estar ocupados por otros entornos de desarrollo.
5432	PostgreSQL	Solo local (127.0.0.1)	**Muy frecuente:** una instalación previa de PostgreSQL en el equipo.
27017	MongoDB	Solo local (127.0.0.1)	Una instalación previa de MongoDB.
6379	Redis	Solo local (127.0.0.1)	Una instalación previa de Redis.

Puerto	Servicio	Expuesto a	Conflicto frecuente con
9092	Kafka	Solo local (127.0.0.1)	Poco frecuente.

Cómo comprobar que un puerto está libre

Windows (PowerShell o CMD):

```
# Reemplace 5432 por el puerto que quiere comprobar
netstat -ano | findstr ":5432"
```

Linux / macOS:

```
lsof -i :5432
```

> **Verificación.** Si el comando **no devuelve ninguna línea**, el puerto está libre. Si devuelve una línea, el puerto está ocupado: identifique el programa y deténgalo, o cambie el puerto en el archivo `.env` (sección 5.3).

3.6 Conectividad

Durante la primera instalación el equipo descarga imágenes y dependencias de internet. Si trabaja detrás de un proxy corporativo o un cortafuegos restrictivo, asegúrese de que tenga acceso a los siguientes dominios:

- `hub.docker.com` y `*.docker.io` — imágenes base de Docker.
- `registry.npmjs.org` — dependencias de los servicios en TypeScript y del frontend.
- `repo.maven.apache.org` — dependencias de los servicios en Java.
- `pypi.org` — dependencias del servicio en Python.
- `proxy.golang.org` — dependencias del servicio en Go.
- `github.com` — descarga del código del proyecto.

4. Descarga e instalación

> Esta sección instala las herramientas base y descarga el proyecto. Al terminar, tendrá el código en su equipo y Docker listo para construirlo.

4.1 Paso 1 — Instalar Docker Desktop

En Windows

1. Abra `https://www.docker.com/products/docker-desktop/` y descargue **Docker Desktop for Windows**.
2. Ejecute el instalador descargado. Cuando pregunte, deje marcada la opción **Use WSL 2 instead of Hyper-V**.
3. Al terminar, **reinicie el equipo** (el instalador lo solicita; es obligatorio para que WSL 2 quede activo).
4. Abra Docker Desktop desde el menú Inicio y espere a que el icono de la ballena, en la barra de tareas, deje de animarse y el panel indique **Engine running**.

En Ubuntu / Debian

```
# 1. Instalar Docker Engine y el plugin de Compose
curl -fsSL https://get.docker.com | sudo sh

# 2. Permitir usar docker sin sudo (evita errores de permisos)
sudo usermod -aG docker $USER

# 3. Cerrar sesión y volver a entrar para que el grupo tenga efecto
```

En macOS

1. Descargue **Docker Desktop for Mac** eligiendo la variante correcta: *Apple Silicon* para procesadores M1/M2/M3, *Intel chip* para los demás.
2. Arrastre Docker a la carpeta Aplicaciones y ábralo.
3. Conceda los permisos que solicite el sistema.

Verificar la instalación de Docker

```
docker --version
docker compose version
docker run hello-world
```

> **Verificación.** Los dos primeros comandos deben mostrar un número de versión (por ejemplo `Docker version 27.3.1` y `Docker Compose version v2.29.7`). El tercero debe terminar con el mensaje *"Hello from Docker! This message shows that your installation appears to be working correctly."* Si ese mensaje aparece, Docker está correctamente instalado.

4.2 Paso 2 — Instalar Git

Sistema	Procedimiento
Windows	Descargar de `git-scm.com/download/win` y ejecutar el instalador aceptando las opciones por defecto.
Ubuntu / Debian	`sudo apt update && sudo apt install -y git`
macOS	`brew install git` (o se instala con las Command Line Tools de Xcode).

```
git --version
```

Verificación. Debe mostrar `git version 2.40.x` o superior.

4.3 Paso 3 — Obtener el proyecto

Existen dos formas de obtener el código. Use la primera si tiene acceso al repositorio.

Opción A — Clonar desde el repositorio (recomendada)

```
# 1. Ubicarse en la carpeta donde desea instalar el proyecto
cd C:\Users\<su-usuario>\Documents        # Windows
cd ~/Documents                             # Linux / macOS

# 2. Clonar el repositorio
git clone https://github.com/Jonas7891/ProyectoFaceAttendEDU.git

# 3. Entrar a la carpeta de código del proyecto
cd ProyectoFaceAttendEDU/FULL
```

Atención — carpeta de trabajo. Todos los comandos de este manual se ejecutan desde la carpeta `FULL/`, que es la raíz del código. Es la única carpeta desde la cual Docker Compose encuentra el archivo `.env` y los archivos de los tres niveles del stack. Si ejecuta los comandos desde otra carpeta, obtendrá errores como `no configuration file provided`.

Opción B — Desde un archivo comprimido

Si recibió el proyecto como archivo `.zip`:

1. Descomprima el archivo en una ruta **sin espacios ni tildes** en el nombre (por ejemplo `C:\proyectos\faceattend`). Las rutas con espacios causan problemas en algunos comandos de Docker.
2. Abra una terminal dentro de la carpeta `FULL` resultante.

4.4 Paso 4 — Verificar la estructura del proyecto

Confirme que la carpeta `FULL/` contiene los siguientes elementos:

```
FULL/
│
├── .env.example                ← plantilla de configuración (se copia en el paso 5)
├── docker-compose.yml          ← orquestador principal (punto de entrada)
├── COMPOSE.md                  ← referencia de infraestructura
│
├── back-end/                   ← los 9 microservicios + la puerta de enlace
│   ├── 01-ms-identity/         Java 21 + Spring Boot
│   ├── 02-ms-authorization/    Java 21 + Spring Boot
│   ├── 03-ms-academic/         TypeScript + Fastify
│   ├── 04-ms-scheduling/       Java 21 + Spring Boot
│   ├── 05-ms-attendance/       Java 21 + Spring Boot
│   ├── 06-ms-biometric/        Python 3.12 + FastAPI
│   ├── 07-ms-configuration/    TypeScript + Fastify
│   ├── 08-ms-notification/     Go 1.22 + Gin
│   ├── 09-ms-quality/          TypeScript + Fastify
│   ├── 99-api-gateway/         Kong OSS 3.6
│   └── docker-compose.yml
│
├── database/                   ← PostgreSQL + las 8 migraciones Liquibase
│   ├── 01-ms-identity-db/ … 08-ms-notification-db/
│   ├── database-init/          crea los 8 esquemas en el primer arranque
│   └── docker-compose.yml
│
├── front-end/
│   ├── Web/                    aplicación web (Expo Web + nginx)
│   └── Mobile/                 aplicación móvil (Expo / React Native)
│
└── tools/
    └── seed-api-test-data/     carga opcional de datos de prueba
```

Comando para verificarlo:

```
dir                     # Windows
ls -la                  # Linux / macOS
```

Verificación. Deben aparecer los archivos `docker-compose.yml` y `.env.example`, y las carpetas `back-end`, `database`, `front-end` y `tools`. Si falta alguno, la descarga quedó incompleta: repita el paso 4.3.

5. Configuración inicial

> Toda la configuración del sistema vive en **un único archivo**: `.env`, ubicado en la raíz de `FULL/`. Docker Compose lo lee automáticamente y distribuye sus valores a los 18 contenedores.

5.1 Paso 5 — Crear el archivo `.env`

El proyecto incluye una plantilla documentada llamada `.env.example`. El primer paso de la configuración es copiarla con el nombre `.env`:

```
# Desde la carpeta FULL/

copy .env.example .env          # Windows (CMD)
Copy-Item .env.example .env     # Windows (PowerShell)
cp .env.example .env            # Linux / macOS
```

Verificación. Ejecute `dir .env` (Windows) o `ls -la .env` (Linux/macOS). El archivo debe existir y pesar aproximadamente 3 KB.

Para una instalación local no hace falta editar nada. Todas las variables tienen valores por defecto que funcionan. Puede continuar directamente a la sección 6 y volver aquí solo si necesita cambiar un puerto o si algo falla.

Atención — el archivo `.env` nunca se sube al repositorio. Está incluido en `.gitignore` por diseño: contiene credenciales. Los valores de la plantilla son credenciales de desarrollo, no secretos reales. Para un entorno de producción, **todas** las contraseñas deben cambiarse y gestionarse mediante variables del sistema operativo o un gestor de secretos, nunca en un archivo del proyecto.

5.2 Variables de entorno

El archivo está organizado por bloques. Esta es la referencia completa:

Base de datos PostgreSQL

Variable	Valor por defecto	Qué controla
`POSTGRES_USER`	`postgres`	Usuario administrador de la base de datos.
`POSTGRES_PASSWORD`	`postgres`	Contraseña de ese usuario. **Cambiar en producción.**
`POSTGRES_DB`	`faceattend_db`	Nombre de la base de datos que contiene los 8 esquemas.

Variable	Valor por defecto	Qué controla
`POSTGRES_PORT`	`5432`	Puerto en el equipo anfitrión. Cámbielo si ya tiene PostgreSQL instalado.
`FACEATTEND_LIQUIBASE_IMAGE`	`liquibase/liquibase:4.29.0`	Imagen que ejecuta las 8 migraciones.

Base de datos MongoDB (biometría)

Variable	Valor por defecto	Qué controla
`MONGO_USER`	`mongoadmin`	Usuario de MongoDB.
`MONGO_PASSWORD`	`mongopass`	Contraseña. **Cambiar en producción.**
`MONGO_DB`	`faceattend_biometric`	Base donde se guardan los vectores faciales y dactilares.
`MONGO_PORT`	`27017`	Puerto en el equipo anfitrión.

Infraestructura de apoyo

Variable	Valor por defecto	Qué controla
`REDIS_PORT`	`6379`	Puerto de Redis (límite de peticiones de Kong).
`KAFKA_PORT`	`9092`	Puerto de Kafka (eventos de dominio).
`KONG_PROXY_PORT`	`8080`	Puerto por el que entran todas las peticiones a la API.
`KONG_ADMIN_PORT`	`8001`	Puerto de administración de Kong (solo local).

Exposición en red e interfaz web

Variable	Valor por defecto	Qué controla
`BIND_IP`	`127.0.0.1`	A qué dirección se asocian los puertos internos. Con `127.0.0.1` solo son accesibles desde el propio equipo; con `0.0.0.0` quedan visibles en toda la red local.
`FRONTEND_PORT`	`8090`	Puerto de la interfaz web.
`EXPO_PUBLIC_API_URL`	`http://localhost:8080`	Dirección de la API que usa el navegador.

> **Atención — `EXPO_PUBLIC_API_URL` se graba durante la construcción.** Esta variable no se lee al arrancar: queda *incrustada* en el paquete de la aplicación web al construir la imagen. Si la cambia, no basta con reiniciar: hay que reconstruir el frontend con `docker compose up -d --build frontend-web`. Es la causa número uno de que la interfaz cargue pero no se conecte con la API.

Usuarios iniciales y datos de prueba

Variable	Valor por defecto	Qué controla
`BOOTSTRAP_USERNAME`	`admin.faceattend`	Usuario administrador creado automáticamente por las migraciones.
`BOOTSTRAP_PASSWORD`	`Admin123!ChangeMe`	Su contraseña. **Cambiar tras el primer inicio de sesión.**
`SEED_USERNAME`	`seed.admin`	Usuario que emplea la herramienta de datos de prueba.
`SEED_PASSWORD`	`SeedAdmin123!`	Su contraseña.

5.3 Casos de personalización

Caso 1 — Ya tengo PostgreSQL instalado y el puerto 5432 está ocupado

Edite `.env` y cambie el puerto en el equipo anfitrión. El puerto interno del contenedor no cambia, así que el resto del sistema sigue funcionando igual:

```
POSTGRES_PORT=5433
```

Caso 2 — Quiero probar desde un celular en la misma red Wi-Fi

Son tres cambios y una reconstrucción. Primero averigüe la IP de su equipo (`ipconfig` en Windows, `ip addr` o `ifconfig` en Linux/macOS), por ejemplo `192.168.1.10`. Luego edite `.env`:

```
BIND_IP=0.0.0.0
EXPO_PUBLIC_API_URL=http://192.168.1.10:8080
```

Y reconstruya el frontend:

```
docker compose up -d --build frontend-web
```

> **Atención.** Kong solo acepta peticiones desde los orígenes `http://localhost:3000`, `:5173` y `:8090`. Para servir desde otra dirección debe agregarla a la lista de orígenes permitidos en `back-end/99-api-gateway/kong/kong.yml`. Además, recuerde volver a `BIND_IP=127.0.0.1` cuando termine la prueba: con `0.0.0.0` las bases de datos quedan expuestas a toda la red local.

Caso 3 — El puerto 8080 está ocupado por otro programa

```
KONG_PROXY_PORT=8070
EXPO_PUBLIC_API_URL=http://localhost:8070
```

Reconstruya después el frontend, por el motivo explicado arriba.

5.4 Canal de correo (opcional)

El servicio de notificaciones puede enviar correos (recuperación de contraseña, verificación de cuenta). Si `SMTP_HOST` queda vacío, el canal se desactiva: el sistema sigue funcionando con normalidad y solo registra un aviso en los registros.

Para probarlo en desarrollo sin un servidor de correo real, levante un buzón falso:

```
docker run -d -p 1025:1025 -p 8025:8025 axllent/mailpit
```

Y configure en `.env`:

```
SMTP_HOST=host.docker.internal
SMTP_PORT=1025
SMTP_FROM=FaceAttend EDU <no-reply@faceattend.local>
SMTP_STARTTLS=false
```

Verificación. Abra `http://localhost:8025`: verá la bandeja donde llegan los correos que envía el sistema.

6. Puesta en marcha

> Con el proyecto descargado y el archivo `.env` creado, el sistema se levanta con un solo comando. La base de datos, las tablas y los datos iniciales se crean automáticamente.

6.1 Paso 6 — Levantar el stack

Desde la carpeta `FULL/`, con Docker Desktop en ejecución:

```
docker compose up -d --build
```

Desglose del comando:

- `docker compose up` — crea y arranca los contenedores definidos.
- `-d` (*detached*) — los deja ejecutándose en segundo plano y le devuelve la terminal.
- `--build` — construye las imágenes de los servicios antes de arrancarlos. Es obligatorio la primera vez.

> **Atención — la primera ejecución es lenta.** Docker debe descargar las imágenes base y compilar los nueve servicios en cuatro lenguajes distintos. Espere entre **15 y 40 minutos** según la velocidad de su conexión y la potencia del equipo. Las ejecuciones siguientes tardan entre 1 y 3 minutos porque reutilizan lo ya construido. No interrumpa el proceso: si lo cancela a medias, vuelva a ejecutar el mismo comando.

Durante la construcción verá líneas como `=> [ms-identity build 3/7] RUN mvn package`. Al terminar, aparecerá un resumen con todos los contenedores creados.

6.2 Orden de arranque

Docker Compose no arranca todo a la vez: respeta una cadena de dependencias para que ningún servicio intente usar algo que todavía no existe.

Fase	Qué arranca	Condición para pasar a la siguiente
1	`postgres`	La base de datos responde correctamente (estado *healthy*).
2	Las 8 migraciones Liquibase, **una después de otra**: identity → authorization → academic → scheduling → attendance → biometric → configuration → notification	Cada una debe terminar con éxito antes de que empiece la siguiente.
3	Los 9 microservicios	Cada uno espera a su propia migración y a que PostgreSQL esté sano.
4	`kong-gateway`	Espera a Redis, Kafka y a los 8 microservicios principales.

Fase	Qué arranca	Condición para pasar a la siguiente
5	`frontend-web`	Espera a que Kong esté sano.

> **Consecuencia práctica.** Si una migración falla, la cadena se detiene y los servicios que dependen de ella no arrancan. Por eso, ante cualquier problema de arranque, lo primero que debe revisarse son los registros de las migraciones (sección 10).

6.3 Base de datos y migraciones

No hay que crear la base de datos ni ejecutar ningún script SQL a mano: el proceso es automático y consta de dos partes.

Creación de los esquemas

La primera vez que PostgreSQL arranca con su volumen de datos vacío, ejecuta el archivo `database/database-init/01-init-schemas.sql`, que crea los ocho esquemas dentro de la base `faceattend_db`: `identity`, `authorization`, `academic`, `scheduling`, `attendance`, `biometric`, `configuration` y `notification`.

> **¿Por qué ocho esquemas y no ocho bases de datos?** Cada microservicio es dueño exclusivo de su esquema y ningún servicio consulta las tablas de otro: las referencias entre contextos se hacen por identificador, sin claves foráneas físicas. Es el patrón *base de datos por contexto*, implementado sobre una sola instancia de PostgreSQL para simplificar el despliegue.

Creación de tablas y datos iniciales

Las ocho tareas de **Liquibase** aplican, en orden, los cambios de cada contexto: primero la estructura (tablas, índices, restricciones) y luego los datos de catálogo imprescindibles. Son contenedores temporales: se ejecutan, terminan y se detienen. Que aparezcan como `Exited (0)` en el listado es el resultado correcto.

En arranques posteriores, Liquibase detecta qué cambios ya se aplicaron y solo ejecuta los nuevos. Por eso es seguro ejecutar `docker compose up -d` todos los días.

6.4 Datos iniciales creados automáticamente

Al terminar las migraciones, el sistema ya contiene lo mínimo para poder usarse:

Roles del sistema

Rol	Alcance
`Administrador`	Acceso total: gestiona usuarios, roles, estructura académica y configuración.
`Instructor`	Docencia: abre y cierra sesiones de clase, registra asistencia y revisa justificaciones.

Rol	Alcance
`Aprendiz`	Consulta de su propia información y envío de justificaciones.

Usuarios iniciales

Usuario	Contraseña inicial	Rol asignado
`admin.faceattend`	`Admin123!ChangeMe`	Administrador
`instructor.faceattend`	`Instructor123!ChangeMe`	Instructor
`aprendiz.faceattend`	`Aprendiz123!ChangeMe`	Aprendiz

> **Atención — seguridad.** Estas credenciales son públicas: están en el código del proyecto y el sufijo `ChangeMe` ("cámbiame") lo indica expresamente. Son aceptables en un entorno de desarrollo o de evaluación. En cualquier instalación accesible por terceros, **cambie las tres contraseñas inmediatamente después del primer inicio de sesión** y desactive las cuentas de demostración que no vaya a usar. El procedimiento está en el *Manual de administrador*.

Además se cargan los catálogos base: la matriz de permisos por rol, los tipos de actor académico, los tipos de justificación y los tipos de alerta.

6.5 Datos de prueba (opcional)

Si necesita el sistema poblado con sedes, programas, fichas, cursos y matrículas de ejemplo —útil para demostraciones o para probar los frontends—, el proyecto incluye una herramienta que los crea mediante llamadas a la API.

Requiere Node.js 20 o superior instalado en el equipo (no usa Docker):

```
# Con el stack ya levantado y todos los servicios sanos
cd tools/seed-api-test-data
npm run seed
```

> **Verificación.** El proceso termina con código de salida `0`. Es seguro ejecutarlo varias veces: si un registro ya existe, lo detecta y continúa.

7. Verificación de la instalación

> No dé por terminada la instalación hasta completar esta sección. Cada prueba comprueba una capa distinta del sistema; ejecútelas en orden, porque un fallo temprano explica los fallos posteriores.

7.1 Lista de verificación

- ☐ Los 18 contenedores aparecen en el listado, los de servicio en estado *running (healthy)*.
- ☐ Las 8 tareas de Liquibase terminaron con `Exited (0)`.
- ☐ Cada microservicio responde a su prueba de salud.
- ☐ La puerta de enlace responde en el puerto 8080.
- ☐ El inicio de sesión devuelve un identificador de sesión válido.
- ☐ La interfaz web carga en el navegador y permite iniciar sesión.
- ☐ La capa de datos no es alcanzable desde la red del navegador.

7.2 Prueba 1 — Estado de los contenedores

```
docker compose ps
```

Resultado esperado. Una tabla con 18 filas. Los microservicios, Kong, el frontend y las bases de datos deben decir `running`, y los que tienen comprobación de salud deben decir `healthy`. Las ocho filas `*-liquibase` deben decir `Exited (0)`: han terminado su trabajo correctamente.

Si algún contenedor dice `restarting`, `unhealthy` o `Exited (1)`, la instalación no está completa. Revise los registros de ese contenedor con `docker compose logs <nombre>` y consulte la sección 10 antes de continuar.

Para ver también los puertos publicados:

```
docker ps --format "{{.Names}} | {{.Status}} | {{.Ports}}"
```

7.3 Prueba 2 — Salud de los servicios

Cada servicio expone una ruta que confirma que está vivo. Las rutas difieren según la tecnología:

```
# Servicios Java (Spring Boot): ruta /actuator/health
curl http://localhost:8081/api/v1/health      # ms-identity
curl http://localhost:8084/actuator/health    # ms-scheduling

# Servicios TypeScript (Fastify): ruta /health
curl http://localhost:8083/health             # ms-academic
curl http://localhost:8087/health             # ms-configuration

# A través de la puerta de enlace (ruta de salud que Kong sí expone)
curl http://localhost:8080/health/quality
```

Resultado esperado. Cada comando devuelve una respuesta en formato JSON con un estado positivo (por ejemplo `{"status":"UP"}` en los servicios Java). Ninguno debe devolver un error de conexión.

Si `curl` no está disponible en Windows, use PowerShell: `Invoke-RestMethod http://localhost:8081/api/v1/health`, o simplemente abra la dirección en el navegador.

7.4 Prueba 3 — Autenticación

Esta es la prueba más importante: recorre toda la cadena (navegador → Kong → ms-identity → PostgreSQL) y confirma que los datos iniciales se cargaron.

```
curl -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d "{\"identifier\":\"admin.faceattend\",\"password\":\"Admin123!ChangeMe\"}"
```

Resultado esperado. Una respuesta JSON con un identificador de sesión:

```
{"sessionId":"a3f1…","userId":"…","sessionStatus":"Active", …}
```

Use ese `sessionId` para comprobar un endpoint protegido:

```
curl http://localhost:8080/api/v1/persons \
  -H "Authorization: Bearer <sessionId-obtenido>"
```

Cómo funciona la autenticación en este sistema. No se usan tokens JWT: al iniciar sesión, el servicio de identidad emite un **identificador de sesión opaco** que se envía en cada petición mediante la cabecera `Authorization: Bearer`. Kong no valida credenciales; cada servicio comprueba la sesión por su cuenta. Por eso, un error 401 nunca viene de la puerta de enlace, sino del servicio de destino.

7.5 Prueba 4 — Interfaz web

1. Abra un navegador actualizado (Chrome, Edge o Firefox).

2. Vaya a `http://localhost:8090`.
3. Debe cargar la pantalla de inicio de sesión de FaceAttend EDU.
4. Inicie sesión con `admin.faceattend` / `Admin123!ChangeMe`.
5. Debe acceder al panel principal correspondiente al rol Administrador.

> **Resultado esperado.** La interfaz carga, acepta las credenciales y muestra el panel. Si la página carga pero el inicio de sesión no responde, abra las herramientas de desarrollo del navegador (tecla **F12**), vaya a la pestaña *Console* y revise si hay errores de conexión: casi siempre indican que `EXPO_PUBLIC_API_URL` no corresponde (vea la sección 10).

Los servicios en Java publican además su documentación interactiva de la API:

- `http://localhost:8081/swagger-ui.html` — identidad
- `http://localhost:8082/swagger-ui.html` — autorización
- `http://localhost:8084/swagger-ui.html` — horarios
- `http://localhost:8085/swagger-ui.html` — asistencia

7.6 Prueba 5 — Aislamiento de red

Esta prueba confirma que la separación en tres redes funciona: el frontend y Kong **no deben poder** alcanzar la base de datos.

```
# Estos DOS comandos deben FALLAR — es el comportamiento correcto
docker exec faceattend-frontend-web nslookup postgres
docker exec faceattend-kong getent hosts postgres

# Este comando debe FUNCIONAR
docker exec faceattend-kong getent hosts ms-identity
```

> **Resultado esperado.** Los dos primeros comandos devuelven un error de resolución de nombre; el tercero devuelve una dirección IP. Si el frontend *sí* resuelve `postgres`, las redes no están correctamente separadas y la base de datos quedaría alcanzable desde la capa expuesta al navegador.

> **Si las cinco pruebas pasaron, la instalación está completa y verificada.** El sistema está listo para usarse. Continúe con el *Manual de usuario* para conocer su funcionamiento, o con el *Manual de administrador* para configurar usuarios reales.

8. Instalación de la aplicación móvil (opcional)

> La aplicación móvil no se despliega con Docker: se ejecuta en modo desarrollo mediante Expo y se abre en un teléfono físico o en un emulador.

8.1 Requisitos adicionales

Requisito	Detalle
Node.js	Versión 20 o superior, instalada en el equipo (`nodejs.org`).
Aplicación Expo Go	Instalada en el teléfono, desde Google Play o la App Store.
Red	El teléfono y el equipo deben estar en la **misma red Wi-Fi**.
Backend	El stack debe estar levantado y accesible desde la red local (`BIND_IP=0.0.0.0`).

8.2 Procedimiento

```
# 1. Averiguar la IP del equipo en la red local
ipconfig            # Windows — busque "Dirección IPv4"
ip addr             # Linux
ifconfig            # macOS

# 2. Entrar a la carpeta de la aplicación móvil
cd front-end/Mobile

# 3. Instalar las dependencias (solo la primera vez, tarda varios minutos)
npm install

# 4. Arrancar el servidor de desarrollo apuntando a la IP del equipo
npx expo start
```

Atención. Desde el teléfono, `localhost` se refiere al propio teléfono, no a su computador. Es obligatorio usar la **IP del equipo en la red local** (por ejemplo `http://192.168.1.10:8080`) en la configuración de la aplicación móvil, y tener `BIND_IP=0.0.0.0` en el archivo `.env` del backend.

1. Al arrancar, Expo muestra un **código QR** en la terminal.
2. Abra **Expo Go** en el teléfono y escanee ese código.
3. La aplicación se descarga al teléfono y arranca.

Verificación. La aplicación abre la pantalla de inicio de sesión y permite entrar con `aprendiz.faceattend` / `Aprendiz123!ChangeMe`. Si la pantalla carga pero el inicio de sesión falla, el problema está en la dirección de la API o en el cortafuegos del equipo, que puede estar bloqueando el puerto 8080.

9. Operación cotidiana

9.1 Comandos frecuentes

Comando	Qué hace
`docker compose up -d`	Arranca el stack reutilizando las imágenes ya construidas (rápido).
`docker compose up -d --build`	Reconstruye las imágenes antes de arrancar. Úselo tras cambiar el código.
`docker compose ps`	Muestra el estado y la salud de cada contenedor.
`docker compose logs -f ms-identity`	Muestra los registros de un servicio en tiempo real (`Ctrl+C` para salir).
`docker compose logs --tail=100 kong-gateway`	Muestra las últimas 100 líneas de registro de un servicio.
`docker compose restart ms-academic`	Reinicia un solo servicio.
`docker compose config`	Valida la configuración sin levantar nada. Devuelve error si hay un problema.
`docker compose down`	Detiene y elimina los contenedores **conservando los datos**.
`docker stats`	Muestra el consumo de CPU y memoria de cada contenedor.

9.2 Detener y reiniciar

```
# Detener el sistema conservando todos los datos
docker compose down

# Volver a arrancarlo (los datos siguen ahí)
docker compose up -d
```

Los datos sobreviven. `docker compose down` elimina los contenedores pero no los volúmenes: la base de datos, los vectores biométricos y los eventos permanecen intactos. Al volver a levantar, el sistema continúa donde quedó.

9.3 Actualizar a una versión nueva

```
# 1. Traer los cambios del repositorio
git pull origin develop

# 2. Revisar si la plantilla de configuración cambió
#    (si aparecen variables nuevas, agréguelas a su .env)

# 3. Reconstruir y arrancar: las migraciones pendientes se aplican solas
docker compose up -d --build
```

9.4 Desinstalar

> **Atención — el siguiente comando borra todos los datos de forma irreversible:** usuarios, asistencias, justificaciones y vectores biométricos. No tiene deshacer. Ejecútelo solo si realmente desea empezar desde cero.

```
# Elimina contenedores Y volúmenes de datos
docker compose down -v

# Elimina además las imágenes construidas (libera ~12 GB)
docker compose down -v --rmi all

# Finalmente, borre la carpeta del proyecto si ya no la necesita
```

10. Solución de problemas (*troubleshooting*)

> Esta sección recoge los problemas más frecuentes, ordenados por el momento en que suelen aparecer. Cada entrada indica el síntoma exacto, la causa y los comandos para resolverlo.

10.1 Antes de diagnosticar: tres comandos

Ante cualquier fallo, ejecute estos tres comandos. Casi siempre identifican la causa:

```
docker compose ps                          # ¿qué contenedor está mal?
docker compose logs --tail=50 <nombre>     # ¿qué dice ese contenedor?
docker compose config                      # ¿la configuración es válida?
```

10.2 Problemas durante la instalación

Síntoma	Causa probable	Solución
`docker: command not found`	Docker no está instalado o no quedó en la variable PATH.	Reinstale Docker Desktop y reinicie el equipo. En Linux, cierre y abra la sesión tras ejecutar `usermod -aG docker`.
`Cannot connect to the Docker daemon`	El servicio de Docker no está en ejecución.	Abra Docker Desktop y espere a que indique *Engine running*. En Linux: `sudo systemctl start docker`.
`no configuration file provided`	Está ejecutando el comando desde una carpeta equivocada.	Ubíquese en la carpeta `FULL/`, donde está `docker-compose.yml`.
La construcción falla descargando dependencias	Sin conexión, o un proxy corporativo bloquea los repositorios.	Verifique la conexión y el acceso a los dominios de la sección 3.6. Reintente: `docker compose build --no-cache`.
`no space left on device`	Disco lleno por imágenes y contenedores antiguos.	`docker system prune -a` libera el espacio de lo que no se está usando. Confirme que quedan al menos 20 GB libres.

10.3 Problemas al arrancar el stack

Error: `port is already allocated` / `bind: address already in use`

Causa: otro programa del equipo ya está usando ese puerto. El caso más frecuente es una instalación previa de PostgreSQL ocupando el 5432.

```
# 1. Identificar qué proceso ocupa el puerto
netstat -ano | findstr ":5432"     # Windows → anote el PID (última columna)
lsof -i :5432                      # Linux / macOS

# 2a. Detener ese proceso…
taskkill /PID <PID> /F             # Windows
kill -9 <PID>                      # Linux / macOS

# 2b. …o bien cambiar el puerto en .env y volver a levantar
#     POSTGRES_PORT=5433
docker compose up -d
```

Error: `service "ms-identity" depends on undefined service "postgres"`

Causa: ejecutó `docker compose up` dentro de la carpeta `back-end/`. Ese archivo por sí solo no define PostgreSQL, que vive en `database/`.

Solución: ejecute siempre desde la raíz `FULL/`. Es un error intencional del diseño, pensado para dejar claro cuál es el punto de entrada.

Un contenedor queda en `restarting` indefinidamente

```
docker compose logs --tail=100 <nombre-del-servicio>
```

Causas habituales: la base de datos aún no estaba lista (se resuelve solo tras unos reintentos), falta una variable de entorno, o el servicio no puede resolver el nombre de otro contenedor. Si tras cinco minutos sigue reiniciándose, reinicie el stack completo:

```
docker compose down && docker compose up -d
```

`frontend-web` aparece como `unhealthy`

Causa conocida: la comprobación de salud debe usar `127.0.0.1` y no `localhost`. Dentro del contenedor, `localhost` también resuelve a la dirección IPv6 `::1`, y nginx solo escucha en IPv4, por lo que la conexión se rechaza.

Solución: el archivo del proyecto ya está corregido. Si ve este síntoma, reconstruya el frontend: `docker compose up -d --build frontend-web`.

10.4 Problemas con las migraciones

Una tarea de Liquibase termina con `Exited (1)`

La cadena de migraciones se detiene y ningún servicio posterior arranca. Identifique cuál falló y por qué:

```
# 1. ¿Cuál falló?
docker compose ps | findstr liquibase      # Windows
docker compose ps | grep liquibase         # Linux / macOS

# 2. ¿Por qué falló? (nombre del changeset y error SQL)
docker compose logs identity-liquibase

# 3. Tras corregir, reejecutar solo esa migración
docker compose up identity-liquibase

# 4. La cadena continúa en el siguiente arranque
docker compose up -d
```

> **Último recurso:** si la base de datos quedó en un estado inconsistente y está en un entorno de pruebas, puede reiniciarla desde cero con `docker compose down -v && docker compose up -d --build`. **Esto borra todos los datos.** Nunca lo ejecute en una instalación con información real.

Error: tabla o esquema no existe

Causa: las migraciones no se ejecutaron o quedaron a medias.

Solución: verifique que las ocho tareas estén en `Exited (0)` y revise los registros de la primera que no lo esté.

10.5 Problemas de conexión y uso

Síntoma	Causa probable	Solución
Pantalla en blanco en `localhost:8090`	El frontend aún no terminó de arrancar, o falló la construcción.	Espere un minuto y recargue. Si persiste: `docker compose logs frontend-web`.
La interfaz carga pero no se conecta con la API	`EXPO_PUBLIC_API_URL` no corresponde a la dirección real de Kong. La variable se graba durante la construcción.	Corrija el valor en `.env` y **reconstruya**: `docker compose up -d --build frontend-web`.
Kong devuelve `502 Bad Gateway`	El microservicio de destino todavía no está sano.	`docker compose ps` para ver cuál falta, y revise sus registros. Espere a que llegue a *healthy*.
Kong devuelve `404 no Route matched`	La ruta solicitada no está declarada en la configuración de Kong.	Compruebe la ruta en el **Anexo B**. Las rutas válidas empiezan por `/api/v1/`.

Síntoma	Causa probable	Solución
`401 Unauthorized` en todas las llamadas	Falta la cabecera de sesión, la sesión expiró, o el usuario no tiene roles asignados.	Obtenga un identificador nuevo con `/api/v1/auth/login` y envíelo como `Authorization: Bearer <sessionId>` en cada petición.
`403 Forbidden`	La sesión es válida pero el rol no tiene el permiso requerido.	Verifique los roles del usuario: `GET /api/v1/users/<userId>/roles`.
Error de CORS en la consola del navegador	Está accediendo desde un origen no autorizado en Kong.	Kong permite `localhost:3000`, `:5173` y `:8090`. Para otro origen, agréguelo en `back-end/99-api-gateway/kong/kong.yml`.
No llegan los correos	`SMTP_HOST` está vacío: el canal está desactivado.	Es el comportamiento esperado por defecto. Para activarlo, configure SMTP según la sección 5.4.

10.6 Problemas de rendimiento

Síntoma	Causa y solución
El equipo se vuelve muy lento tras levantar el stack	18 contenedores, cuatro de ellos con máquina virtual de Java, consumen memoria de forma considerable. Cierre otras aplicaciones y asigne más memoria a Docker Desktop (*Settings → Resources → Memory*, al menos 6 GB). Con `docker stats` puede ver qué contenedor consume más.
La construcción tarda más de una hora	Normal en la primera ejecución con conexión lenta: se descargan unos 5 GB. Las siguientes son mucho más rápidas. No interrumpa el proceso.
Contenedores que se detienen solos	Docker se quedó sin memoria asignada (error *OOMKilled*). Aumente la memoria de Docker Desktop o levante solo los servicios que necesite.

10.7 Reinicio limpio

Si tras agotar las opciones anteriores el sistema sigue sin funcionar, este procedimiento deja todo como una instalación nueva:

```
# 1. Detener y eliminar todo, incluidos los datos
docker compose down -v

# 2. Limpiar recursos de Docker no utilizados
docker system prune -a

# 3. Verificar que .env existe y es correcto
#    (si hay dudas, vuelva a copiarlo de .env.example)

# 4. Reconstruir desde cero
docker compose up -d --build
```

Atención. Este procedimiento borra todos los datos del sistema. Úselo solo en entornos de desarrollo o evaluación.

11. Instalación sin Docker (avanzado)

> **Esta sección es opcional y no se recomienda para una instalación normal.** Solo tiene sentido si va a modificar el código de un servicio y quiere depurarlo desde su entorno de desarrollo. Para instalar y usar el sistema, el procedimiento con Docker de las secciones 4 a 7 es más rápido y mucho menos propenso a errores.

11.1 Herramientas por lenguaje

Cada servicio requiere su propio entorno instalado en el equipo:

Lenguaje	Versión	Servicios afectados	Dónde obtenerlo	Verificación
Java (JDK)	21 LTS	identity, authorization, scheduling, attendance	adoptium.net	`java --version`
Maven	3.9+	los mismos cuatro	maven.apache.org	`mvn --version`
Node.js	20+	academic, configuration, quality, ambos frontends	nodejs.org	`node --version`
Python	3.12	biometric	python.org	`python --version`
Go	1.22	notification	go.dev	`go version`

11.2 Enfoque recomendado: modo híbrido

Lo habitual no es prescindir de Docker, sino levantar con Docker toda la infraestructura (bases de datos, Kafka, Kong y los servicios que no está tocando) y ejecutar fuera de Docker únicamente el servicio en el que está trabajando:

```
# 1. Levantar solo la capa de datos y las migraciones
docker compose up -d database

# 2. Ejecutar un servicio desde su carpeta, según el lenguaje
cd back-end/01-ms-identity   && mvn spring-boot:run        # Java
cd back-end/03-ms-academic   && npm install && npm run dev # TypeScript
cd back-end/06-ms-biometric  && uvicorn main:app --reload  # Python
cd back-end/08-ms-notification && go run ./cmd/server      # Go
```

> **Los servicios leen el mismo archivo `.env`.** Al arrancar fuera de Docker, cada servicio carga la configuración desde la raíz del proyecto: los servicios Java mediante `spring.config.import`, los de TypeScript con `process.loadEnvFile` y el de Python con una función propia en `settings.py`. No hay que duplicar la configuración.

11.3 Frontend web en modo desarrollo

```
cd front-end/Web
npm install
npx expo start --web
```

Arranca en `http://localhost:8081` con recarga automática al guardar cambios.

12. Glosario

> Términos técnicos empleados en este manual, explicados en el contexto de este proyecto.

Término	Definición
Contenedor	Paquete que incluye un programa y todo lo que necesita para funcionar (sistema base, librerías, configuración). Se ejecuta aislado del resto del equipo, de modo que funciona igual en cualquier máquina.
Imagen	Plantilla de solo lectura a partir de la cual se crean los contenedores. Construir una imagen es compilar el servicio y empaquetarlo; arrancar un contenedor es ejecutar una copia de esa imagen.
Docker	Plataforma que construye y ejecuta contenedores.
Docker Compose	Herramienta que levanta varios contenedores relacionados con un solo comando, respetando el orden de dependencias descrito en el archivo `docker-compose.yml`.
Volumen	Espacio de disco gestionado por Docker donde los contenedores guardan datos que deben sobrevivir a su eliminación (en este proyecto: la base de datos, los vectores biométricos y los eventos).
Microservicio	Servicio independiente, responsable de una sola parte del negocio, con su propia base de datos y su propio ciclo de despliegue. Este sistema tiene nueve.
API Gateway / puerta de enlace	Componente que recibe todas las peticiones externas y las dirige al microservicio correspondiente. Aquí lo cumple Kong, en el puerto 8080. Aplica además CORS y límite de peticiones.
Kong	Puerta de enlace de código abierto usada en el proyecto. Se configura de forma declarativa mediante el archivo `kong.yml`, sin base de datos propia.
Endpoint	Dirección concreta de la API que atiende una operación, por ejemplo `POST /api/v1/auth/login`.
Esquema (*schema*)	Subdivisión lógica dentro de una base de datos PostgreSQL. Este sistema usa una sola base (`faceattend_db`) dividida en ocho esquemas, uno por contexto.
Migración	Cambio versionado en la estructura de la base de datos (crear una tabla, agregar una columna). Se aplican en orden y quedan registradas, de modo que nunca se ejecutan dos veces.
Liquibase	Herramienta que ejecuta las migraciones. En este proyecto son ocho tareas, una por esquema, que se ejecutan automáticamente al levantar el stack.
Changeset	Unidad individual de cambio dentro de una migración de Liquibase.

Término	Definición
Datos semilla (*seed*)	Registros mínimos que el sistema necesita para arrancar: roles, permisos, tipos de justificación y el usuario administrador inicial.
Variable de entorno	Valor de configuración que se define fuera del código (aquí, en el archivo `.env`) y que el programa lee al arrancar. Permite cambiar puertos o credenciales sin tocar el código.
Puerto	Número que identifica un canal de comunicación en un equipo. Dos programas no pueden usar el mismo puerto a la vez.
Loopback / `127.0.0.1`	Dirección que apunta al propio equipo. Un servicio asociado a ella solo es accesible desde esa máquina, no desde la red.
Healthcheck	Comprobación periódica que Docker ejecuta sobre un contenedor para determinar si está funcionando. De ahí los estados *healthy* y *unhealthy*.
CORS	Mecanismo de seguridad del navegador que bloquea las peticiones a un dominio distinto del de la página, salvo que ese dominio las autorice expresamente.
Sesión opaca	Identificador sin contenido legible que el servidor entrega al iniciar sesión y que el cliente envía en cada petición. Este sistema lo usa en lugar de tokens JWT.
RBAC	*Role-Based Access Control*: control de acceso basado en roles. Los permisos se asignan a roles, y los roles a usuarios.
Kafka	Sistema de mensajería que permite a los microservicios comunicarse de forma asíncrona mediante eventos, sin llamarse directamente.
Redis	Almacén de datos en memoria. Aquí guarda los contadores del límite de peticiones de Kong.
MongoDB	Base de datos orientada a documentos. Aquí almacena los vectores biométricos faciales y dactilares.
Expo	Plataforma sobre React Native que permite construir con un mismo código la aplicación web y la móvil.
nginx	Servidor web que entrega al navegador los archivos ya construidos de la interfaz.
WSL 2	*Windows Subsystem for Linux*: capa de Windows que permite ejecutar Linux. Docker Desktop lo usa para correr los contenedores.

13. Soporte y referencias

13.1 Documentación del proyecto

Antes de escalar un problema, revise la documentación interna, que es la fuente autorizada:

Documento	Contenido
`FULL/COMPOSE.md`	Referencia completa de la infraestructura Docker: archivos, orden de arranque, redes, puertos y variables.
`FULL/.env.example`	Plantilla comentada de todas las variables de configuración.
`FULL/back-end/SERVICES.md`	Guía general de los microservicios: responsabilidades, tecnologías y dependencias.
`FULL/database/SEEDS.md`	Detalle de los datos iniciales: roles, permisos y usuarios de arranque.
`fae-docs/10-devops/local-setup.md`	Guía de configuración del entorno local para desarrolladores.
`fae-docs/05-architecture/`	Decisiones de arquitectura (ADR) y diagramas.
`fae-docs/07-api/`	Contratos de la API, autenticación y catálogo de errores.

13.2 Documentación externa

Herramienta	Dirección
Docker	`docs.docker.com`
Docker Compose	`docs.docker.com/compose`
PostgreSQL 17	`postgresql.org/docs/17`
Liquibase	`docs.liquibase.com`
Kong Gateway	`docs.konghq.com`
Expo	`docs.expo.dev`

13.3 Canal de soporte

Dato	Valor
Repositorio del proyecto	`github.com/Jonas7891/ProyectoFaceAttendEDU`
Reporte de incidencias	Sección *Issues* del repositorio
Equipo responsable	_____
Correo de contacto	_____
Instructor(a) a cargo	_____

14. Anexos

Anexo A — Mapa completo de puertos

Puerto	Servicio	Expuesto a	Dirección de acceso
8090	frontend-web	0.0.0.0	`http://localhost:8090`
8080	kong-gateway (proxy)	0.0.0.0	`http://localhost:8080`
8001	kong-gateway (admin)	`BIND_IP`	`http://localhost:8001`
8081	ms-identity	`BIND_IP`	`http://localhost:8081/swagger-ui.html`
8082	ms-authorization	`BIND_IP`	`http://localhost:8082/swagger-ui.html`
8083	ms-academic	`BIND_IP`	`http://localhost:8083/health`
8084	ms-scheduling	`BIND_IP`	`http://localhost:8084/actuator/health`
8085	ms-attendance	`BIND_IP`	`http://localhost:8085/actuator/health`
8086	ms-biometric	`BIND_IP`	`http://localhost:8086/docs`
8087	ms-configuration	`BIND_IP`	`http://localhost:8087/health`
8088	ms-notification	`BIND_IP`	`http://localhost:8088`
8089	ms-quality	`BIND_IP`	`http://localhost:8089`
5432	PostgreSQL	`BIND_IP`	`localhost:5432` — base `faceattend_db`
27017	MongoDB	`BIND_IP`	`localhost:27017`
6379	Redis	`BIND_IP`	`localhost:6379`
9092	Kafka	`BIND_IP`	`localhost:9092`

Anexo B — Rutas principales de la API

Todas pasan por la puerta de enlace en `http://localhost:8080` y requieren la cabecera `Authorization: Bearer <sessionId>`, salvo el inicio de sesión.

Servicio de destino	Rutas
Identidad	`/api/v1/auth`, `/api/v1/persons`, `/api/v1/users`, `/api/v1/sessions`, `/api/v1/cities`, `/api/v1/password-policies`

Servicio de destino	Rutas
Autorización	`/api/v1/roles`, `/api/v1/permissions`, `/api/v1/auth/evaluate`, `/api/v1/users/{id}/roles`
Académico	`/api/v1/schools`, `/api/v1/programs`, `/api/v1/academic-periods`, `/api/v1/cohorts`, `/api/v1/courses`, `/api/v1/academic-actors`, `/api/v1/enrollments`
Horarios	`/api/v1/environments`, `/api/v1/schedule-blocks`, `/api/v1/class-sessions`
Asistencia	`/api/v1/attendance-records`, `/api/v1/justifications`, `/api/v1/justification-types`, `/api/v1/supporting-documents`
Biometría	`/api/v1/biometric`
Configuración	`/api/v1/academic-configurations`, `/api/v1/security-configurations`, `/api/v1/biometric-update-cases`
Notificaciones	`/api/v1/alert-types`, `/api/v1/alerts`
Calidad	`/api/v1/quality`, `/health/quality`

Anexo C — Credenciales iniciales

> **Estas credenciales son públicas y de desarrollo.** Cámbielas en cuanto la instalación sea accesible por terceros.

Usuario	Contraseña	Rol
`admin.faceattend`	`Admin123!ChangeMe`	Administrador
`instructor.faceattend`	`Instructor123!ChangeMe`	Instructor
`aprendiz.faceattend`	`Aprendiz123!ChangeMe`	Aprendiz

Anexo D — Formato de reporte de incidencias

Si un problema no se resuelve con la sección 10, repórtelo con esta información. Sin estos datos, el diagnóstico remoto es prácticamente imposible:

```
REPORTE DE INCIDENCIA — INSTALACIÓN FACEATTEND EDU

1. Datos del entorno
   Sistema operativo y versión : _____________________________
   Memoria RAM del equipo      : _____________________________
   Versión de Docker           : (salida de `docker --version`)
   Versión de Docker Compose   : (salida de `docker compose version`)

2. Punto de la instalación donde ocurrió
   Sección y paso del manual   : _____________________________
   Comando ejecutado           : _____________________________

3. Comportamiento
   Resultado esperado          : _____________________________
   Resultado obtenido          : _____________________________
   Mensaje de error completo   : _____________________________

4. Evidencia adjunta
   [ ] Salida de `docker compose ps`
   [ ] Salida de `docker compose logs --tail=100 <servicio-afectado>`
   [ ] Captura de pantalla del error
   [ ] Contenido del archivo .env (SIN las contraseñas)

5. Acciones ya intentadas
   ____________________________________________________________
```

Anexo E — Resumen de la instalación en seis comandos

Para quien ya conoce el procedimiento y solo necesita la secuencia:

```
# 1. Obtener el proyecto
git clone https://github.com/Jonas7891/ProyectoFaceAttendEDU.git

# 2. Entrar a la carpeta de código
cd ProyectoFaceAttendEDU/FULL

# 3. Crear la configuración
cp .env.example .env

# 4. Levantar todo el stack (15–40 min la primera vez)
docker compose up -d --build

# 5. Verificar el estado
docker compose ps

# 6. Abrir la interfaz
#    http://localhost:8090 → admin.faceattend / Admin123!ChangeMe
```